Diseño e implementación de un modelo NoSQL con Redis para reserva de entradas bajo alta concurrencia

Resumen

Este trabajo presenta el diseño e implementación de una solución basada en un modelo NoSQL clave-valor para el caso de la EntradaFlash CR una plataforma ficticia de ventas de entradas la cual opera en periodos de alta demanda. Se utilizó Redis como tecnología de almacenamiento para administrar la disponibilidad de las entradas, las reservas temporales y el control de intentos. La solución incluye mecanismos para manejar reservas con vencimiento, las operaciones simultaneas y evitar inconsistencias en el inventario. Además, se realizaron pruebas funcionalidad de concurrencia, rendimiento y persistencia para evaluar el desempeño del sistema. Los resultados permiten analizar el desempeño de la solución y las alternativas de persistencia ante posibles fallos del servicio.

Palabras claves: NoSQL, clave-valor, Redis, concurrencia, reservas temporales.

I. Introducción

EntradaFlash CR es una plataforma de venta de entradas para conciertos, partidos y espectáculos que debe operar durante períodos de alta demanda. Debido a la gran cantidad de usuarios, es común que miles de personas realicen solicitudes de manera simultánea, ya sea para consultar la disponibilidad de entradas, realizar reservas o cancelar sus compras. Por este motivo, manejar correctamente las operaciones simultáneas es fundamental para este tipo de plataformas, ya que una gestión inadecuada puede resultar en bloqueos, tiempos de respuesta elevados o incluso en la venta de localidades que ya fueron reservadas por otro usuario [1].

El problema al cual se enfrenta EntradaFlash CR consiste en diseñar una capa de datos que trabaje de manera rápida y eficiente y que permita administrar la disponibilidad de las entradas, las reservas temporales y el estado de las sesiones de los usuarios bajo condiciones de alta concurrencia. De esta manera, se busca evitar que dos o más usuarios puedan reservar la misma entrada al mismo tiempo y garantizar que el inventario disponible se mantenga correctamente.

Para abordar este problema se seleccionó el modelo NoSQL clave-valor, debido a que permite almacenar y consultar la información de manera sencilla y directa. Este modelo resulta adecuado para la plataforma, ya que permite organizar la información necesaria para manejar las entradas, las reservas y las sesiones de los usuarios [1].

El objetivo de este trabajo es diseñar e implementar una solución basada en el modelo clave-valor que permita administrar las entradas de EntradaFlash CR de manera eficiente durante períodos de alta demanda. Además, se busca comprobar el funcionamiento de la solución mediante pruebas de concurrencia, disponibilidad, reservas

temporales, manejo de sesiones y rendimiento [6].

ll. Fundamentos del modelo NoSQL

Las bases de datos NoSQL son sistemas de almacenamiento que emplean modelos distintos al modelo relacional tradicional. Estos sistemas pueden ser adecuados para aplicaciones que manejan una gran cantidad de información y que necesitan procesar varias solicitudes de manera rápida y simultánea. Además, los modelos NoSQL pueden organizar los datos de diferentes formas, como documentos, grafos, columnas o pares clave-valor [1].

En este caso en particular, se implementa el modelo clave-valor para organizar la información, el cual consta, como su nombre lo indica, de una clave y un valor. La clave permite identificar un dato específico, mientras que el valor contiene la información asociada a dicha clave. Una de las principales ventajas de este modelo es que permite acceder directamente a la información utilizando la clave, lo que resulta conveniente cuando se requiere consultar o modificar un dato específico [1].

En EntradaFlash CR, este modelo se encarga de organizar la información necesaria para el funcionamiento de la plataforma. Por ejemplo, permite almacenar la cantidad de entradas disponibles mediante una clave que identifica un evento y una zona determinada [6].

Otra característica importante para este proyecto es la posibilidad de manejar información temporal. En una plataforma como EntradaFlash CR, una entrada que ha sido reservada no necesariamente será comprada, por lo que es necesario contar con mecanismos que establezcan un tiempo de duración para este tipo de información. El modelo clave-valor también es útil cuando varios usuarios realizan operaciones simultáneamente, ya que la tecnología utilizada debe permitir manejar estas operaciones de forma segura y evitar inconsistencias en el inventario [3], [4].

En este proyecto se utiliza Redis como sistema de almacenamiento basado en el modelo clave-valor. Sus características permiten trabajar con información almacenada mediante claves, establecer tiempos de expiración y realizar operaciones de manera conjunta y segura. Estas funciones son importantes para implementar las reservas temporales y controlar las operaciones realizadas de manera simultánea en EntradaFlash CR [2]–[4].

lll. Tecnología seleccionada

Para la implementación de la solución se escogió Redis como la tecnología de almacenamiento. Redis es un sistema de almacenamiento de datos que utiliza el modelo clave-valor y permite acceder rápidamente a la información almacenada. Esto resulta adecuado para la EntradaFlash CR, debido a que la plataforma debe de responder a un gran número de solicitudes en periodos de alta demanda [2].

Una de las características de Redis favorables para el proyecto es la posibilidad de establecer tiempos de expiración de los datos. Esto permite manejar las reservas temporales de las entradas, pues estas deben dejar de estar vigentes luego de un periodo de tiempo determinado. De ese modo se evita que la

entrada quede reservada, aunque el cliente no complete el proceso [3].

Redis permite ejecutar ciertas operaciones de forma atómica, es decir, como una sola operación que no puede quedar parcialmente ejecutada. Esto es importante para EntradaFlash CR ya que varios usuarios pueden intentar reservar entradas al mismo tiempo. Para manejar estas situaciones Redis cuenta con scripts Lua los cuales permiten comprobar si hay entradas disponibles y actualizar el inventario en una sola operación. De esta manera, se evita que dos o más usuarios reserven la misma entrada [4],[6].

Otra característica bastante útil para el proyecto que tiene Redis es permite almacenar distintos tipos de datos mediante claves. Esto deja la posibilidad organizar por separado los datos relacionados con el inventario, las reservas, los carritos de los usuarios y el control de intentos. De esta forma, la información necesaria para cada operación puede ser consultada directamente [2], [6].

Finalmente, Redis cuenta con mecanismos de persistencia que permiten conservar los datos ante posibles fallos o reinicio del servicio [5].

IV. Modelo y Arquitectura

Para la EntradaFlash CR se diseñó una estructura de datos basada en el modelo clave-valor, utilizando Redis como la tecnología de almacenamiento de la información relacionada con las entradas, reservas y usuarios. La información se organiza mediante claves únicas, lo cual permite identificar los datos necesarios para cada operación sin almacenar toda la información en una única estructura [2], [6].

Una de las claves utilizadas corresponde al inventario de las entradas disponibles para cada zona y evento identificada mediantes event: {evento}:zone:{zona}:stock. Esta clave permite consultar y actualizar la cantidad de entradas disponibles. Por otra parte, las reservas se identifican mediante reservation:{usuario}:{evento}: {zona} donde se almacena el estado de la reserva ya sea pendiente, confirmada o cancelada [6].

Además de eso, se implementaron claves para administrar los carritos de los usuarios y para controlar la cantidad de intentos realizados. Los carritos se identifican mediante cart:{usuario}, mientras que los intentos se registran mediante rate_limit:{usuario} y rate_limit:ip:{ip}. También se almacena la información de los usuarios con la clave user:{usuario}, Esta organización permite separar los datos según su función y facilita su consulta en las operaciones de la plataforma [6].

Por otra parte, en cuanto al funcionamiento del sistema Redis se encarga de almacenar y actualizar información mientras que los módulos desarrollados en Python ejecutan las operaciones requeridas por la plataforma. El módulo correspondiente a las reservas permite consultar la disponibilidad, crear reservas temporales y confirmarlas o cancelarlas. Asimismo, utiliza scripts Lua para comprobar la disponibilidad y actualizar el inventario, evitando que las solicitudes simultáneas generen inconsistencias. Similarmente, módulo de sesiones administra los carritos de los usuarios y el módulo de control de intentos limita las solicitudes por usuario y dirección IP [4], [6].

Finalmente, para preparar el escenario de evaluación se desarrolló un proceso de generación y carga de datos que contempla cinco eventos, cada uno con tres zonas, 50 000 usuarios y 100 000 intentos de reserva. Estos datos permiten evaluar el comportamiento de la solución en un escenario simulado de alta demanda [6].

V. Implementación

La solución se implementó utilizando Redis y Python. El código se organizó por módulos para separar las operaciones relacionadas con las reservas, los carritos de los usuarios y el control de intentos. Esta organización facilita la ejecución de las pruebas al mantener las distintas funciones del sistema separadas [6].

Para preparar los datos, se desarrolló un proceso de generación y carga que crea cinco eventos cada uno con tres zonas, 50 000 usuarios y 100 000 intentos de reserva. Seguidamente, los datos se cargan en Redis mediante un script de inicialización. Para ejecutar el proyecto, se usa una instancia de Redis en el puerto 6380 [6].

En cuanto a las reservas, el sistema permite consultar las entradas disponibles y realizar reservas temporales. Estas reservas tienen un período de vigencia y cuando vencen, el sistema contempla la devolución de las entradas al inventario. Además, se pueden confirmar o cancelar las reservas según el estado de la operación. Para proteger el inventario frente a solicitudes simultáneas, se utilizan scripts Lua que permiten ejecutar las operaciones necesarias para comprobar la disponibilidad y actualizar las cantidades [3], [4], [6].

El módulo de sesiones permite mantener un carrito asociado a cada usuario, mientras que el módulo de control de intentos limita las solicitudes realizadas por usuario y dirección IP. Estas funciones complementan el manejo de las reservas y permiten controlar tanto las sesiones temporales como la frecuencia de las solicitudes del sistema [6].

Finalmente, se realizan pruebas automatizadas, demostraciones funcionales y scripts para evaluar la concurrencia y medir el rendimiento del sistema. Estas herramientas permiten verificar las funciones implementadas y analizar el comportamiento de la solución en diferentes escenarios [6].

VI. Pruebas

Para evaluar la solución implementada en EntradaFlash CR, se realizaron pruebas funcionales de concurrencia, rendimiento y persistencia. Estas evaluaciones permiten evaluar el funcionamiento de las operaciones principales y analizar su comportamiento bajo diferentes situaciones [6].

Pruebas funcionales

Se evaluaron las operaciones de consulta de disponibilidad, creación de reservas temporales, confirmación y cancelación de reservas. También se comprobó el funcionamiento de los carritos de los usuarios y del control de intentos por usuario y dirección IP. Estas pruebas tenían el objetivo de verificar que las operaciones se ejecutaran de acuerdo con las condiciones establecidas para la plataforma [6].

Pruebas de concurrencia

Se evaluó el comportamiento del inventario cuando varios usuarios

intentaban reservar entradas al mismo tiempo. La prueba permitió comprobar si las operaciones atómicas implementadas mediante scripts Lua evitaban que se reservaran más entradas de las disponibles y que el inventario alcanzara valores negativos [4], [6].

Pruebas de rendimiento

Se realizaron pruebas para medir el rendimiento de operaciones de lectura y escritura, así como los tiempos de respuesta. Se consideraron las condiciones de ejecución y el nivel de concurrencia utilizados en cada evaluación [6].

Pruebas de persistencia y recuperación

Se evaluó el comportamiento de Redis ante posibles interrupciones del servicio utilizando distintas configuraciones de persistencia. Se compararon los mecanismos RDB y AOF, así como la ejecución sin persistencia, para observar qué información podía recuperarse después de una caída y cómo afectaba cada configuración al rendimiento [5], [6].

Latencia

La latencia permite analizar el tiempo que requiere el sistema para ejecutar las operaciones evaluadas. La Tabla II presenta los tiempos de respuesta obtenidos durante las pruebas, lo que permite observar el comportamiento de Redis al procesar las solicitudes [6].

Tabla ll.

Latencia (1 cliente, mediana de las repeticiones)

VII. Resultados

Rendimiento

En esta sección se presentan los resultados obtenidos en las pruebas de rendimiento de EntradaFlash CR. Se analiza la capacidad del sistema para procesar operaciones bajo las condiciones de carga evaluadas. La Tabla I resume los resultados obtenidos y permite comparar el rendimiento de las operaciones medidas [6].

Las pruebas de rendimiento evaluaron la capacidad del sistema para procesar operaciones bajo distinta concurrencia. El sistema alcanzó un rendimiento cercano a 25 000 operaciones por segundo con 64 clientes concurrentes, manteniendo latencias medianas inferiores a 0.1 ms en las operaciones clave de lectura y escritura. Las transacciones mantuvieron una tasa de éxito del 100% en todos los escenarios evaluados. Todo esto es evidenciado en el anexo B.

Operación	Tipo	Req.	Promedio (ms)	p50 (ms)	p95 (ms)	p99 (ms)	ops/s	Éxito
GET stock (Redis directo)	lectura	base	0.082	0.081	0.092	0.102	12,140	100 %
SET clave (Redis directo)	escritura	base	0.086	0.085	0.092	0.102	11,692	100 %
DECRBY stock (Redis directo)	atómica	base	0.082	0.082	0.088	0.092	12,196	100 %
consultar_disponibilidad	lectura	R1	0.080	0.080	0.085	0.088	12,536	100 %
crear_reserva_temporal	escritura + atómica	R2/R4	0.098	0.098	0.105	0.140	10,192	100 %
confirmar_reserva	atómica	R3/R4	0.087	0.086	0.096	0.111	11,560	100 %
cancelar_reserva	atómica	R3/R4	0.095	0.095	0.100	0.126	10,577	100 %
agregar_item (carrito)	escritura + atómica	R5	0.188	0.189	0.201	0.214	5,308	100 %
ver_carrito	lectura	R5	0.090	0.090	0.094	0.117	11,111	100 %
verificar_intento (usuario + IP)	atómica	R6	0.095	0.095	0.101	0.121	10,490	100 %

Carrito y control de intentos

Para el carrito, se validó la gestión multi-zona con un tope de 6 entradas: al intentar agregar 5 ítems teniendo 3 VIP previas, la operación fue rechazada sin afectar el

inventario (stock en 1000). Los carritos se almacenan en Redis con un TTL de 900 s y fueron recuperados íntegramente por otra conexión tras simular una caída. Al finalizar la compra, las entradas se convirtieron en reservas temporales (el stock bajó de 1000 a 998 en VIP y de 1500 a 1499 en PREMB) y las zonas sin disponibilidad registraron su rechazo sin pérdidas ni duplicados. Bajo concurrencia, 50 pestañas de un mismo usuario respetaron el tope [6].

Para el control de intentos, 200 solicitudes simultáneas de un usuario frente a un límite de 20 resultaron en 20 permitidas y 180 rechazadas en ~400 ms, sin contar los rechazos. La validación y el incremento se ejecutan de forma atómica mediante scripts Lua. Una prueba con 100 usuarios Las pruebas de concurrencia permitieron evaluar el comportamiento de las reservas cuando varios usuarios intentan adquirir entradas de forma simultánea. Se busca corroborar que las operaciones se ejecuten correctamente y que la disponibilidad del inventario se mantenga consistente, incluso cuando existen solicitudes simultáneas [4], [6].

Se realizó una prueba de caída la cual evidencia la diferencia en el estado del sistema antes del fallo y tras el reinicio. Antes de la interrupción, se registraban 72,570 claves totales, 10 ventas confirmadas, 5 reservas pendientes activas, 3 entradas en carrito abierto y un stock de 70 en la zona demo. Tras el reinicio del servicio, el total de claves disminuyó a 72,535, mientras que las ventas confirmadas, reservas pendientes y carritos cayeron a 0, perdiéndose además las referencias del stock de la zona demo, Esto se aprecia mejor en la Tabla III.

tras una IP con límite de 15 permitió exactamente 15 accesos. El proyecto cuenta con 53 pruebas automáticas (22 de carrito, 14 de control de intentos y 17 de reservas), todas aprobadas.

Concurrencia, caída y persistencia

En las pruebas de alta concurrencia con 100 000 intentos de reserva, se vendieron en total 22 497 entradas, y se rechazaron 76 638 por falta de stock y 1 295 por restricciones de usuario. El sistema mantuvo un stock mínimo de cero y ninguna reserva pendiente sin liberar, por lo que el inventario fue consistente, sin sobreventa ni stock negativo en todas las zonas. Esto se demuestra en el anexo A.

Tabla lll.

Prueba de caída

Dato	Antes de la caída	Después de reiniciar
Claves totales en Redis	72,570	72,535
Dataset (usuarios de prueba presentes, de 2)	2	2
Stock VIP EVT-101	1000	1000
Ventas confirmadas (de 10)	10	0
Reservas pendientes vivas (de 5)	5	0
Stock zona demo	70	None
Entradas en el carrito abierto (req. 5)	3	0
Stock zona demo tras vencer las pendientes (correcto: 80)	-	None

Por otra parte, se compararon distintas configuraciones de persistencia de Redis para analizar la conservación de los datos ante una caída del servicio y el efecto de estas configuraciones sobre el rendimiento. La Tabla IV presenta los resultados de esta comparación [5], [6].

Tabla IV.

Comparativa de persistencia

Operación	sin-persistencia p50 / p99 (ms)	aof-everysec p50 / p99 (ms)	aof-always p50 / p99 (ms)
GET stock (Redis directo)	0.081 / 0.088	0.077 / 0.091	0.077 / 0.091
SET clave (Redis directo)	0.083 / 0.091	0.084 / 0.108	3.997 / 5.032
DECRBY stock (Redis directo)	0.081 / 0.089	0.082 / 0.109	3.995 / 4.152
consultar_disponibilidad	0.077 / 0.088	0.077 / 0.101	0.081 / 0.104
crear_reserva_temporal	0.102 / 0.118	0.112 / 0.159	3.999 / 4.949
confirmar_reserva	0.085 / 0.115	0.102 / 0.136	4.000 / 5.546
cancelar_reserva	0.097 / 0.132	0.103 / 0.141	4.002 / 6.222
agregar_item (carrito)	0.196 / 0.223	0.205 / 0.233	4.946 / 9.046
ver_carrito	0.092 / 0.119	0.092 / 0.121	0.086 / 0.109
verificar_intento (usuario + IP)	0.098 / 0.125	0.105 / 0.133	4.000 / 6.005

Los resultados permiten comparar las alternativas evaluadas según la conservación de los datos y el costo de rendimiento asociado a cada configuración. Se analiza la diferencia entre utilizar RDB y AOF con las opciones *everysec* y *always*, y ejecutar Redis sin persistencia [5], [6].

Vll. Conclusiones

El desarrollo de EntradaFlash CR permitió implementar una solución basada en el modelo NoSQL clave-valor utilizando Redis para gestionar la disponibilidad de las entradas, las reservas temporales y las sesiones de los usuarios. La solución implementa mecanismos para controlar las operaciones simultáneas y mantener la consistencia del inventario mediante el uso de scripts Lua [4], [6].

Por otra parte, la comparación de persistencia permitió analizar diferencias entre las configuraciones evaluadas de Redis. Al no haber persistencia se dio la pérdida de los datos tras la caída del servicio, mientras que RDB permitió recuperar la información correspondiente al último punto de guardado. En la prueba realizada con AOF y la configuración *everysec* no se observaron pérdidas de datos, mientras que *always* presentó un mayor costo de rendimiento [5], [6].

Los resultados permiten identificar un equilibrio entre rendimiento y conservación de los datos. Para el escenario evaluado, AOF con *everysec* resulta ser una alternativa a considerar, ya que presentó un menor impacto en el rendimiento que *always* y permitió recuperar los datos en la prueba realizada.

Las pruebas de rendimiento permitieron evaluar la capacidad del sistema para procesar operaciones con diferentes cantidades de clientes concurrentes. En la evaluación, se alcanzó un rendimiento de alrededor de 25 000 operaciones por segundo con 64 clientes concurrentes en lo que respecta a lecturas. Por otra parte, con reservas se llegó a las 21 400 operaciones y a 10 900 con carrito. De igual forma, las mediciones de latencia registraron tiempos de respuesta aproximados de 0,1 ms en las operaciones evaluadas. Estos resultados muestran el comportamiento de la solución bajo las condiciones específicas de las pruebas realizadas [6].

No obstante, la selección definitiva depende de los requisitos de disponibilidad y recuperación de la plataforma [5], [6].

IX. Limitaciones

Los resultados obtenidos corresponden a un entorno de pruebas en específico, por lo que no necesariamente representan el rendimiento que tendría EntradaFlash CR en un entorno de producción distinto. Además, las mediciones se realizaron en un equipo MacBook con procesador M3 y 16 GB de memoria RAM, utilizando una versión determinada de Redis por lo que otros equipos, versiones y condiciones de carga podrían producir resultados diferentes [6].

Adicionalmente, las pruebas de rendimiento y latencia se limitaron a las operaciones y condiciones contempladas

en el experimento. Debido a esto, los valores obtenidos no deben interpretarse como una garantía de rendimiento general para cualquier cantidad de usuarios o patrón de solicitudes [6].

En cuanto a la persistencia, las conclusiones se basan en los escenarios de caída evaluados. Aunque las pruebas permiten comparar el comportamiento de las configuraciones utilizadas, no garantizan que se obtenga el mismo resultado ante cualquier tipo de fallo o interrupción. Sería necesario contemplar otras evaluaciones para estudiar otros escenarios de recuperación [5], [6].

Finalmente, los resultados no permiten evaluar por completo el comportamiento de una plataforma comercial pues esta también dependería de otros componentes, como los sistemas de pago, la infraestructura de red y los servicios que interactúan con la capa de datos.

X. Referencias

[1] S. Ramzan, I. S. Bajwa, R. Kazmi y Amna, "Challenges in NoSQL-Based Distributed Data Storage: A Systematic Literature Review," *Electronics*, vol. 8, n.º 5, art. 488, abr. 2019, doi: 10.3390/electronics8050488.

[2] Redis Ltd., "Redis data types," *Redis Documentation*. [En línea]. Disponible en: https://redis.io/docs/latest/develop/data-types/. [Consultado: 8-oct-2026].

[3] Redis Ltd., "EXPIRE," *Redis Documentation*. [En línea]. Disponible en: https://redis.io/docs/latest/commands/expire/. [Consultado: 8-oct-2026].

[4] Redis Ltd., "Scripting with Lua," *Redis Documentation*. [En línea]. Disponible en: https://redis.io/docs/latest/develop/programmability/eval-intro/. [Consultado: 8-oct-2026].

[5] Redis Ltd., "Redis persistence," *Redis Documentation*. [En línea]. Disponible en: https://redis.io/docs/latest/management/persistence/. [Consultado: 8-oct-2026].

[6] F. Fang et al., "EntradaFlash CR con Redis," GitHub, repositorio del proyecto. [En línea]. Disponible en: https://github.com/fabiolafang/EntradaFlash_Redis. [Consultado: 8-oct-2026].

XI. Anexos

Anexo A: Rendimiento de clientes recurrentes

Operación	Hilos	ops/s	p50 (ms)	p99 (ms)	Éxito
GET stock (Redis directo)	1	10,795	0.086	0.100	100.0 %
GET stock (Redis directo)	8	24,888	0.275	0.700	100.0 %
GET stock (Redis directo)	32	24,907	1.032	3.692	100.0 %
GET stock (Redis directo)	64	24,493	2.042	7.429	100.0 %
consultar_disponibilidad	1	10,481	0.091	0.105	100.0 %
consultar_disponibilidad	8	24,139	0.282	0.713	100.0 %
consultar_disponibilidad	32	25,543	1.045	3.579	100.0 %
consultar_disponibilidad	64	24,624	2.048	8.208	100.0 %
crear_reserva_temporal	1	8,710	0.107	0.154	100.0 %
crear_reserva_temporal	8	21,147	0.323	0.809	100.0 %
crear_reserva_temporal	32	21,949	1.169	5.092	100.0 %
crear_reserva_temporal	64	21,378	2.265	10.356	100.0 %
confirmar_reserva	1	9,405	0.100	0.135	100.0 %
confirmar_reserva	8	23,321	0.298	0.734	100.0 %
confirmar_reserva	32	23,809	1.077	4.194	100.0 %
confirmar_reserva	64	22,939	2.136	9.855	100.0 %
agregar_item (carrito)	1	4,859	0.200	0.229	100.0 %
agregar_item (carrito)	8	10,787	0.680	1.419	100.0 %
agregar_item (carrito)	32	11,093	2.580	7.207	100.0 %
agregar_item (carrito)	64	10,869	5.129	15.015	100.0 %
verificar_intento (usuario + IP)	1	9,220	0.102	0.130	100.0 %
verificar_intento (usuario + IP)	8	22,033	0.314	0.809	100.0 %
verificar_intento (usuario + IP)	32	22,127	1.152	4.513	100.0 %
verificar_intento (usuario + IP)	64	22,208	2.260	10.470	100.0 %

Anexo B: Concurrencia

```
Motivos de rechazo al reservar:
  Stock insuficiente.                                    76,638
  El usuario ya tiene una compra confirmada en esta zona.     1,219
  El usuario ya tiene una reserva pendiente en esta zona.        76

Zona                  Inicial  Vendidas   Final  Inicial-Vendidas  Cuadra
EVT-101 GENERAL          2000      1999       1                 1      si
EVT-101 PREMB            1500      1500       0                 0      si
EVT-101 VIP              1000      1000       0                 0      si
EVT-102 GENERAL          2000      2000       0                 0      si
EVT-102 PREMB            1500      1500       0                 0      si
EVT-102 VIP              1000      1000       0                 0      si
EVT-103 GENERAL          2000      2000       0                 0      si
EVT-103 PREMB            1500      1500       0                 0      si
EVT-103 VIP              1000       999       1                 1      si
EVT-104 GENERAL          2000      1999       1                 1      si
EVT-104 PREMB            1500      1500       0                 0      si
EVT-104 VIP              1000      1000       0                 0      si
EVT-105 GENERAL          2000      2000       0                 0      si
EVT-105 PREMB            1500      1500       0                 0      si
EVT-105 VIP              1000      1000       0                 0      si

Stock mínimo observado durante la prueba: 0
Reservas pendientes sin liberar al final: 0
Total vendido: 22,497 entradas (según Redis: 22,497)

VEREDICTO: inventario consistente, sin sobreventa ni stock negativo

Nota: esta prueba vendió entradas reales del dataset. Ejecute seed_redis.py para restaurar el inventario original.

Esta prueba vendio entradas. Abre 2_PREPARAR_DATOS.bat para restaurar el stock.
Presione una tecla para continuar . . .
```